SIMATS ENGINEERING

CSA4712 – DEEP LEARNING
ASSESSMENT 4

ANALYSE, COMPARE AND ENGINEER LATENT-VARIABLE AND GRAPHICAL MODELS

Student Name	B Akshaya
Registration Number	192424336
Submission Date	31.10.2026
Scenario	Smart-Hospital Monitoring Platform

Assignment overview

This assignment analyses linear latent-variable models and structured probabilistic models for a smart-hospital monitoring platform. It presents the mathematical derivations, numerical calculations, model comparisons, implementation approaches and independent verification checkpoints for all nine tasks. Numerical examples are illustrative or synthetic where stated and should not be interpreted as clinical diagnoses.

TASK 1 – Deconstruct a Linear Latent-Variable Model of Patient Sensors (4 Marks)

A. Derive the observation model (1 Mark)

Assume $x = Wh + \varepsilon$, where $h \sim N(0, I)$ and $\varepsilon \sim N(0, \Psi)$, independently. Then x is Gaussian:

$p(x) = N(0, WW^T + \Psi)$, $\quad Cov(x) = WW^T + \Psi$.

Illustrative one-factor loading vector: $W = [0.8, 0.6, 0.7, 0.2]^T$. Let $\Psi = diag(0.36, 0.64, 0.51, 0.96)$. The diagonal noise values are chosen so each observed sensor has unit variance.

Sensor	Loading w_i	Shared variance w_i^2	Unique variance ψ_{ii}
S1 – Heart rate	0.8	0.64	0.36
S2 – Skin temperature	0.6	0.36	0.64
S3 – Oxygen saturation	0.7	0.49	0.51
S4 – Room temperature	0.2	0.04	0.96

WW^T contains the shared covariances. Adding Ψ gives:

$\Sigma = WW^T + \Psi =$

[[1.00, 0.48, 0.56, 0.16],\ [0.48, 1.00, 0.42, 0.12],\ [0.56, 0.42, 1.00, 0.14],\ [0.16, 0.12, 0.14, 1.00]]

For example, $Cov(S1,S3) = 0.8 \times 0.7 = 0.56$. The diagonal term for S1 is $0.8^2 + 0.36 = 1.00$. Since Ψ is diagonal, it contributes only to the variances, not to cross-sensor covariances.

B. Shared and sensor-specific variance (1 Mark)

Communality is $h_i = w_i^2$ and uniqueness is ψ_{ii}. S1 has communality 0.64, S2 has 0.36, S3 has 0.49 and S4 has 0.04. S1 is most strongly explained by the shared latent factor, followed by S3. S4 has the largest uniqueness (0.96), suggesting room temperature is strongly affected by independent environmental/HVAC variation rather than the shared physiological factor.

C. Conditional independence (1 Mark)

Graph: h → S1, h → S2, h → S3 and h → S4. With diagonal Ψ, the sensor noises are independent, so S1, S2, S3 and S4 are conditionally independent given h. Marginally they remain correlated because they share the common parent h. For S1 and S3, covariance = $w_1 w_3 = 0.56$. If Ψ has off-diagonal entries, residual sensor noises are correlated and conditional independence no longer generally holds.

D. Posterior inference (1 Mark)

For the one-dimensional latent variable, posterior variance is $V = (1 + W^T\Psi^{-1}W)^{-1}$ and posterior mean is $\mu = V W^T\Psi^{-1}x$. For $x = [1.1, 0.7, -0.4, 0.9]^T$, the illustrative model gives $V = 0.2303$ and $\mu = 0.6307$. Sensor contributions to the posterior mean are approximately S1: +0.5629, S2: +0.1511, S3: −0.1264 and S4: +0.0432; their sum is 0.6307. Sensors with lower uniqueness have greater precision ($1/\psi_{ii}$), so their observations have more influence.

Self-verification: Generate 50,000 samples with NumPy default_rng(21), $h \sim N(0,1)$ and $\varepsilon \sim N(0,\Psi)$. Compare the sample covariance with Σ above using max(abs(S_sample − Σ)). Sampling error should be small and decrease as sample size increases.

```
rng = np.random.default_rng(21)\ n = 50000\ h = rng.normal(size=(n, 1))\ eps = rng.multivariate_normal(np.zeros(4), psi, size=n)\ X = h @ W\.T + eps\ S_sample = np.cov(X, rowvar=False)\ print(np.max(np.abs(S_sample - (W @ W\.T + psi))))
```

TASK 2 – Analyse Probabilistic PCA for Patient-State Compression (6 Marks)

A. Select the latent dimension (2 Marks)

Using the illustrative covariance matrix Σ from Task 1, its eigenvalues in descending order are approximately 2.0330, 0.9434, 0.5925 and 0.4311. In PPCA, σ^2ML is the mean of the discarded eigenvalues. The variance fraction represented by the retained subspace is the sum of retained eigenvalues divided by total variance (4.0000).

Latent dimension M	σ^2ML	Variance represented
1	0.6557	50.8248%
2	0.5118	74.4105%
3	0.4311	89.2234%

For M = 2, loading-vector lengths are $\sqrt{(\lambda_1 - \sigma^2)} = \sqrt{(2.0330 - 0.5118)} \approx 1.2334$ and $\sqrt{(\lambda_2 - \sigma^2)} = \sqrt{(0.9434 - 0.5118)} \approx 0.6570$. M = 2 is a reasonable compromise: it retains about 74.41% of the total variance with fewer latent dimensions than the original four sensors. M = 3 retains more variance but provides less compression.

B. Model complexity (1 Mark)

Excluding the mean and accounting for rotational freedom, the PPCA count is $dM - M(M-1)/2 + 1$, including the isotropic noise variance. For d = 4, the counts are M=1: 5, M=2: 8, and M=3: 10. An unrestricted symmetric 4 × 4 covariance matrix has $d(d+1)/2 = 10$ parameters. PPCA is more compact for M=1 or 2, but a lower parameter count can mean less flexibility.

C. Posterior uncertainty (1 Mark)

For PPCA, p(h|x) is Gaussian with covariance $C = \sigma^2(W^TW + \sigma^2I)^{-1}$ and mean $W^T(WW^T + \sigma^2I)^{-1}(x-\mu)$. Under M=2, the posterior variances along the two principal latent directions are approximately $\sigma^2/\lambda_1 = 0.2517$ and $\sigma^2/\lambda_2 = 0.5426$. The second latent direction is less certain because its eigenvalue is smaller. Unlike deterministic PCA, PPCA provides a posterior distribution and uncertainty. As σ^2 approaches zero, the posterior becomes increasingly concentrated around the PCA projection, assuming nonzero retained eigenvalues.

D. Implement and verify PPCA (2 Marks)

The following code implements closed-form PPCA from the sample covariance. It reports the maximum-likelihood noise variance, loading matrix and covariance reconstruction error. The orthogonal rotation check verifies that W and WR describe the same covariance because $(WR)(WR)^T = WW^T$.

```
import numpy as np\ from sklearn.decomposition import PCA\ \ def fit_ppca(X, M):\ mu = X.mean(axis=0)\ S = np.cov(X, rowvar=False)\ vals, U = np.linalg.eigh(S)\ order = np.argsort(vals)[::-1]\ vals, U = vals[order], U[:, order]\ d = X.shape[1]\ sigma2 = vals[M:].mean()\ W = U[:, :M] @ np.diag(np.sqrt(np.maximum(vals[:M] - sigma2, 0)))\ S_hat = W @ W\.T + sigma2 \ np.eye(d)\ err = np.linalg.norm(S - S_hat, ord="fro")\ return mu, vals, sigma2, W, S_hat, err\ \ rng = np.random.default_rng(31)\ W_true = rng.normal(size=(5, 2))\ H = rng.normal(size=(1200, 2))\ E = rng.normal(0, 0.4, size=(1200, 5))\ X = H @ W_true.T + E\ mu, vals, s2, W_ml, S_hat, err = fit_ppca(X, 2)\ print("sigma2_ML:", s2)\ print("W_ML:\n", W_ml)\ print("Frobenius error:", err)\ theta = np.deg2rad(30)\ R1 = np.array([[np.cos(theta), -np.sin(theta)],\ [np.sin(theta), np.cos(theta)]])\ theta2 = np.deg2rad(60)\ R2 = np.array([[np.cos(theta2), -np.sin(theta2)],\ [np.sin(theta2), np.cos(theta2)]])\ print(np.linalg.norm(W_ml @ W_ml.T - (W_ml @ R1) @ (W_ml @ R1).T))\ print(np.linalg.norm(W_ml @ W_ml.T - (W_ml @ R2) @ (W_ml @ R2).T))\ pca = PCA(n_components=2).fit(X)\ print("PCA noise variance:", pca.noise_variance_)\ for n in [60, 250, 1200]:\ # Regenerate each sample size with the same
```

data-generating model\ rg = np.random.default_rng(31)\ Hn = rg.normal(size=(n, 2))\ En = rg.normal(0, 0.4, size=(n, 5))\ Xn = Hn @ W_true.T + En\ print(n, fit_ppca(Xn, 2)[2], fit_ppca(Xn, 2)[5])

Interpretation: W is not unique because W and WR give the same covariance for any orthogonal R. Small samples usually produce noisier covariance eigenvalues and larger estimation error. The scikit-learn noise variance should agree with the closed-form estimate up to numerical conventions and the use of sample covariance.

TASK 3 – Differentiate Factor Analysis from Probabilistic PCA (6 Marks)

A. Predict before computing (2 Marks)

Before calculation, the first principal component is expected to give large weight to sensors that contribute strongly to total covariance, while Factor Analysis can give a larger loading to a sensor with a strong common-factor relationship even if its unique noise differs. For the illustrative model, S1 has the largest absolute loading (0.8), followed by S3 (0.7). The first principal direction from the covariance eigen-decomposition is approximately $[-0.5970, -0.5160, -0.5675, -0.2350]^T$; the sign is arbitrary, so its absolute weights are compared. This is consistent with S1 and S3 receiving the strongest weights.

B. Experimental comparison (2 Marks)

Generate 2,500 observations using default_rng(41), the illustrative W and Ψ from Task 1. A representative run gives PPCA $\sigma^2_{ML} \approx 0.6494$ and one-factor FA uniqueness estimates approximately [0.3328, 0.6379, 0.5125, 0.9527]. The FA estimates are close to the chosen noise levels [0.36, 0.64, 0.51, 0.96]. PPCA uses one shared isotropic noise value, while FA estimates a separate uniqueness for each sensor; FA can therefore represent unequal sensor noise more faithfully.

C. Rotational ambiguity (1 Mark)

For a two-factor matrix W and an orthogonal rotation R(30°), W_rot = WR. Since $RR^T = I$, W_rot $W_{rot}^T = W RR^T W^T = WW^T$. The covariance and likelihood remain unchanged even though individual loadings change. Varimax is sometimes used after fitting to obtain a simpler, more interpretable loading pattern.

D. Sensitivity to measurement units (1 Mark)

Multiplying oxygen saturation by 100 changes its variance and covariance by a factor of 10,000 and 100 respectively. Without standardisation, PCA/PPCA may be dominated by that column and noise estimates become difficult to compare. Standardise each feature using training-set means and standard deviations before comparing sensors measured in different units. In FA, the rescaled loading and uniqueness should transform consistently with the change of units; this can be checked numerically.

\# Illustrative simulation and FA cross-check\ import numpy as np\ from sklearn.decomposition import FactorAnalysis\ rng = np.random.default_rng(41)\ n = 2500\ W = np.array([[0.8], [0.6], [0.7], [0.2]])\ psi_diag = np.array([0.36, 0.64, 0.51, 0.96])\ H = rng.normal(size=(n, 1))\ E = rng.normal(size=(n, 4)) \ np.sqrt(psi_diag)\ X = H @ W\.T + E\ S = np.cov(X, rowvar=False)\ vals, U = np.linalg.eigh(S)\ vals = vals[::-1]; U = U[:, ::-1]\ sigma2 = vals[1:].mean()\ print("PPCA sigma2:", sigma2)\ fa = FactorAnalysis(n_components=1, random_state=41).fit(X)\ print("FA uniqueness:", fa.noise_variance_)\ \# A custom EM implementation can be compared with this library cross-check.\ \# Standardise before comparing features with different physical units.

Articulation checkpoint: Rescale only the oxygen-saturation column, refit FA, and compare the rescaled loading/uniqueness after converting back to the original units. Small differences are expected because of finite-sample estimation.

TASK 4 – Recover Slow Physiological Trends Using Slow Feature Analysis (5 Marks)

A. Measure raw sensor dynamics (1 Mark)

For a reproducible illustrative dataset, define slow(t)=sin(2πt/500), fast(t)=sin(2πt/10), x_1=slow+fast+noise and x_2=slow−fast+noise. With 2,000 samples and noise SD 0.5, a representative run gives Var(x_1) ≈ 1.2649 and Var(x_2) ≈ 1.2516. Slowness is Δ(x)=mean[(x(t+1)−x(t))²]; the values are about 0.7001 for x_1 and 0.6776 for x_2. Thus x_2 is slightly slower in this run. Variance measures spread, whereas slowness measures temporal change, so the two rankings need not match.

B. Identify the hidden slow source (1 Mark)

$y_1=(x_1+x_2)/2$ approximately cancels the fast source and retains slow(t); $y_2=(x_1−x_2)/2$ approximately cancels the slow source and retains fast(t). For the illustrative run, Δ(y_1) ≈ 0.2370 and Δ(y_2) ≈ 0.4519. The slow trend is hidden in both raw sensors because each contains a mixture of slow and fast sources.

C. Contrast SFA with PCA (1 Mark)

PCA diagonalises the covariance matrix and orders components by decreasing variance. SFA first centres and whitens the observations, then diagonalises the covariance of temporal differences and selects the eigenvectors with the smallest eigenvalues. Therefore, PCA maximises variance, while SFA minimises the average squared temporal derivative under zero-mean, unit-variance and decorrelation constraints. A maximum-variance component is not necessarily the slowest component.

D. Implement SFA and analyse noise sensitivity (2 Marks)

A simple linear SFA procedure is shown below. With noise SD 0.5, 1.0 and 2.0, the illustrative first two SFA slowness values are approximately [0.3805, 0.7103], [0.9548, 1.1973] and [1.5319, 1.6874]. The maximum absolute correlation with the underlying slow/fast sources falls from about 0.895 at SD 0.5 to about 0.71 at SD 1.0 and 0.46 at SD 2.0. Thus, recovery deteriorates as noise becomes comparable to or larger than the signal.

```
import numpy as np\ \ def run_sfa(noise_sd, seed=11, n=2000):\ rng = np.random.default_rng(seed)\ t = np.arange(n)\ slow = np.sin(2\np.pi\t/500)\ fast = np.sin(2\np.pi\t/10)\ X = np.column_stack([\ slow + fast + rng.normal(0, noise_sd, n),\ slow - fast + rng.normal(0, noise_sd, n)\ ])\ X = X - X.mean(axis=0)\ C = np.cov(X, rowvar=False)\ ev, U = np.linalg.eigh(C)\ Z = X @ U @ np.diag(1/np.sqrt(ev)) @ U.T # whiten\ dZ = np.diff(Z, axis=0)\ Cd = np.cov(dZ, rowvar=False)\ evd, V = np.linalg.eigh(Cd) # ascending = slowest first\ Y = Z @ V\ slowness = np.mean(np.diff(Y, axis=0)\\2, axis=0)\ corr_slow = [abs(np.corrcoef(Y[:,i], slow)[0,1]) for i in range(2)]\ corr_fast = [abs(np.corrcoef(Y[:,i], fast)[0,1]) for i in range(2)]\ return Y, slowness, corr_slow, corr_fast\ \ for sd in [0.5, 1.0, 2.0]:\ Y, sl, cs, cf = run_sfa(sd)\ print(sd, sl, cs, cf)\ print("means:", Y.mean(axis=0), "variances:", Y.var(axis=0))\ C_y = np.cov(Y, rowvar=False)\ print("max off-diagonal covariance:", abs(C_y[0,1]))
```

Self-verification: The outputs should have means near zero, variances near one and small off-diagonal covariance. Numerical deviations occur because the sample is finite and because whitening/temporal differences use estimated covariance matrices.

TASK 5 – Engineer Sparse Representations for Rare Clinical Events (5 Marks)

Illustrative inputs

The assignment refers to a supplied orthonormal dictionary and observation, but they are not included in the document text. For a worked example, use D = I_3 and x = $[1.2, 0.5, 0.1]^T$. For this dictionary, the optimal sparse code is the soft-thresholded vector h\ = sign(x) max(|x|−λ, 0).

A. Effect of the sparsity penalty (1 Mark)

λ	Code h\	Reconstruction error \	\	x−Dh\	\	₂	L1 norm	Non-zero count
0.2	[1.0, 0.3, 0.0]	0.2236	1.3000	2				
0.8	[0.4, 0.0, 0.0]	0.7141	0.4000	1				
1.8	[0.0, 0.0, 0.0]	1.3038	0.0000	0				

The smallest all-zero threshold is $\lambda \geq \max|D^T x| = 1.2$. Among the tested values, $\lambda = 1.8$ produces the all-zero code. Increasing λ encourages sparsity but increases reconstruction error.

B. Compare alternative representations (1 Mark)

For an overcomplete illustrative dictionary with atoms $d_1=[1,0]^T$, $d_2=[0,1]^T$ and $d_3=[1,1]^T/\sqrt{2}$, observation $x=[1,1]^T$ has dense representation h_dense=$[1,1,0]^T$ and sparse representation h_sparse=$[0,0,\sqrt{2}]^T$. Both reconstruct x exactly. Dense code: $L_0=2$ and $L_1=2$. Sparse code: $L_0=1$ and $L_1\approx1.4142$. An overcomplete dictionary permits multiple representations; a sparsity penalty favours a representation using fewer atoms.

C. One ISTA iteration (1 Mark)

For $D=I_3$, $L=\lambda\max(D^TD)=1$. Starting from $h^{(0)}=0$ with $\lambda=0.3$, the ISTA step is $h^{(1)}=\text{soft}(h^{(0)} + D^T(x-Dh^{(0)})/L, \lambda/L)=[0.9,0.2,0.0]^T$. The objective $J(h)=0.5||x-Dh||^2_2+\lambda||h||_1$ is $J(0)=0.8500$ and $J(h^{(1)})=0.4250$. The first atom is selected because $|x_1|$ is the largest correlation with the observation.

D. Implement ISTA and evaluate support recovery (2 Marks)

For the illustrative identity dictionary, the converged support is obtained by soft-thresholding. $\lambda=0.01$ recovers all three coefficients but retains a small coefficient likely to be noise; $\lambda=0.05$ and 0.2 retain the first two coefficients; $\lambda=0.8$ retains only the first; $\lambda=0.8$ can miss a real but weak atom. Compare with the minimum-norm pseudoinverse code $h=D^+x=x$, which has zero reconstruction error but is not sparse.

import numpy as np\ \ def soft_threshold(z, t):\ return np.sign(z) \ np.maximum(np.abs(z) - t, 0.0)\ \ def ista(D, x, lam, max_iter=10000, tol=1e-10):\ L = np.linalg.eigvalsh(D.T @ D).max()\ h = np.zeros(D.shape[1])\ history = []\ for _ in range(max_iter):\ h_new = soft_threshold(h + D.T @ (x - D @ h) / L, lam / L)\ J = 0.5\np.linalg.norm(x - D @ h_new)\\2 + lam\np.linalg.norm(h_new, 1)\ history.append(J)\ if np.linalg.norm(h_new-h) < tol:\ h = h_new\ break\ h = h_new\ return h, history\ \ D = np.eye(3)\ x = np.array([1.2, 0.5, 0.1])\ for lam in [0.01, 0.05, 0.2, 0.8]:\ h, hist = ista(D, x, lam)\ print(lam, h, np.linalg.norm(x-D\@h), np.linalg.norm(h,1),\ np.count_nonzero(np.abs(h)>1e-8), hist[-1])\ print("monotone objective:", np.all(np.diff(hist) <= 1e-9))

Optimality check: for nonzero h_i, the gradient of the reconstruction term should equal $-\lambda\ \text{sign}(h_i)$; for zero coefficients, its magnitude should not exceed λ (within numerical tolerance).

TASK 6 – Analyse a Directed Patient-State Graphical Model (5 Marks)

A. Factorise the joint distribution (1 Mark)

The graph is Activity A → HeartRate H; Activity A → RoomState R; H → Alert L; and R → Alert L. Its joint distribution factorises as:

P(A,H,R,L) = P(A) P(H|A) P(R|A) P(L|H,R).

For four binary variables, an unrestricted joint distribution has $2^4-1 = 15$ free parameters. The directed model has 1 parameter for P(A), 2 for P(H|A), 2 for P(R|A), and 4 for P(L|H,R), for a total of 9 free parameters.

B. Exact inference (1 Mark)

The assignment says to use instructor-supplied CPTs, but these CPTs are absent from the DOCX. For a worked example, use P(A=High)=0.30; P(H=High|A=High)=0.80 and P(H=High|A=Low)=0.20; P(R=Unstable|A=High)=0.40 and P(R=Unstable|A=Low)=0.10. Set P(L=High|H,R) to 0.95 for high heart rate and unstable room, 0.80 for high heart rate and stable room, 0.70 for low heart rate and unstable room, and 0.05 for low heart rate and stable room.

Quantity	Illustrative exact result	
P(Alert = High)	0.4035	
P(HeartRate = High)	0.3800	
P(Alert = High \	Activity = High)	0.7500
P(Alert = High \	Activity = High, RoomState = Unstable)	0.9000

The posterior alert probability rises from 0.75 to 0.90 after observing an unstable room, because the added evidence is directly related to alert risk through the graph.

C. Ancestral sampling (1 Mark)

Sample nodes in topological order A, H, R, L. For each node, draw a uniform random number u and assign the high/unstable state if u is below the relevant conditional probability. For four samples, record the uniform draw, the conditional probability used and the resulting state at every node. The empirical high-alert rate is the number of high alerts divided by four; it will be noisy because four samples is very small.

D. Sampling and convergence (2 Marks)

Ancestral sampling can estimate P(Alert=High) directly. Rejection sampling estimates P(Alert=High | evidence) by retaining only samples that match the evidence. As N increases from 10 to 100,000, the estimate generally stabilises and absolute error usually decreases, although it need not decrease monotonically for every random seed. Rare evidence produces a low acceptance rate and wastes many samples.

```
import numpy as np\ \ def ancestral_sample(rng):\ A = int(rng.random() < 0.30)\ pH = 0.80 if A else 0.20\ H = int(rng.random() < pH)\ pR = 0.40 if A else 0.10\ R = int(rng.random() < pR) # 1 = unstable\ pL = {(1,1):0.95, (1,0):0.80, (0,1):0.70, (0,0):0.05}[(H,R)]\ L = int(rng.random() < pL)\ return A, H, R, L\ \ rng = np.random.default_rng(61)\ for N in [10, 100, 1000, 10000, 100000]:\ samples = np.array([ancestral_sample(rng) for _ in range(N)])\ print(N, "P(Alert=High) estimate =", samples[:,3].mean())\ mask = (samples[:,0] == 1) & (samples[:,2] == 1)\ accepted = samples[mask]\ estimate = accepted[:,3].mean() if len(accepted) else float("nan")\ print("conditional estimate =", estimate, "retained =", len(accepted),\ "acceptance =", len(accepted)/N)
```

Self-verification: At N=100,000, compare the estimated high-alert probability with the exact illustrative value 0.4035. The conditional dependence should show higher alert risk when activity is high and the room is unstable.

TASK 7 – Use Gibbs Sampling for an Undirected Sensor Dependency Model (5 Marks)

A. Exact enumeration (1 Mark)

For a three-node binary chain $x_i \in \{-1,+1\}$ with pairwise coupling J and no external fields, $P(x) \propto \exp[J(x_1x_2+x_2x_3)]$. There are $2^3=8$ states. The partition function is $Z = 2e^{(2J)}+4+2e^{(-2J)}$. The all-positive

state has probability e^(2J)/Z. For illustrative J=0.7, Z≈12.078 and P(+++)≈0.1887. A 250-node binary graph has $2^{250} \approx 1.81\times10^{75}$ states, so exact enumeration is impractical.

B. Gibbs conditional (1 Mark)

For an interior node i with neighbours x_{i-1} and x_{i+1}, $P(x_i=+1 \mid \text{rest}) = 1/[1+\exp(-2J(x_{i-1}+x_{i+1}))]$. The partition function cancels because it is constant for the two candidate values of x_i. For J=0.7, neighbours (+1,+1) give 0.9427; (+1,−1) gives 0.5000; (−1,−1) gives 0.0573.

C. Trace two Gibbs sweeps (1 Mark)

The instructor's starting state and any stated random sequence are not present in the provided document. A valid trace must begin with that state, visit nodes in the specified order, calculate the conditional probability above at each update, compare it with the next uniform draw and record the updated state. If no sequence is supplied, fix a seed and state it so the trace is reproducible.

D. Implement and analyse mixing (2 Marks)

Run Gibbs sampling from two different starting states and compare empirical distributions or total-variation distance at 100, 1,000 and 10,000 sweeps. Burn-in is adequate when summaries from different initial states become similar and remain stable. Stronger coupling (J=1.8) makes neighbouring states align more strongly and usually slows mixing; sign changes in the mean state become less frequent, so samples are more correlated.

```
import numpy as np\ from itertools import product\ \ def exact_chain3(J):\ states = np.array(list(product([-1, 1], repeat=3)))\ weights = np.exp(J\(states[:,0]\states[:,1] + states[:,1]\states[:,2]))\ probs = weights / weights.sum()\ p_all_positive = probs[np.all(states == 1, axis=1)][0]\ return states, probs, weights.sum(), p_all_positive\ \ def gibbs_chain(n_nodes, J, sweeps, seed, initial=None):\ rng = np.random.default_rng(seed)\ x = np.ones(n_nodes, dtype=int) if initial is None else np.array(initial, dtype=int)\ samples = []\ for _ in range(sweeps):\ for i in rng.permutation(n_nodes):\ neigh = 0\ if i > 0: neigh += x[i-1]\ if i < n_nodes-1: neigh += x[i+1]\ p_plus = 1 / (1 + np.exp(-2\J\neigh))\ x[i] = 1 if rng.random() < p_plus else -1\ samples.append(x.copy())\ return np.asarray(samples)\ \ states, probs, Z, ppp = exact_chain3(0.7)\ print("Z, P(+++) =", Z, ppp)\ for J in [0.7, 1.8]:\ samples = gibbs_chain(12, J, 10000, seed=7)\ mean_state = samples.mean(axis=1)\ print(J, "mean-state sign changes:", np.sum(np.diff(np.sign(mean_state)) != 0))
```

Self-verification: Compare the empirical all-positive frequency from the three-node chain after 10,000 sweeps with the exact value. The discrepancy should be consistent with finite-sample error and autocorrelation.

TASK 8 – Compare Models and Select the Appropriate Technique (6 Marks)

A. Contrast the models (2 Marks)

Method	Latent assumption / noise	Optimised quantity	Ambiguity / invariance	Limitation								
PPCA	Gaussian latent factors; isotropic noise	Gaussian likelihood / covariance fit	W may rotate orthogonally	Equal-noise assumption can be unrealistic								
Factor Analysis	Gaussian factors; separate sensor uniqueness	Likelihood	Factors may rotate	Linear model may miss nonlinear dependencies								
SFA	Outputs are slowly varying	Temporal-difference variance	Sign/order and sometimes	Sensor drift can be mistaken for								

	after whitening		rotations	a slow feature								
Sparse Coding	Signal is a sparse combination of dictionary atoms	0.5\	\	x−Dh\	\	² + λ \	\	h \	\	₁	Overcomplete dictionaries allow multiple codes	λ causes shrinkage and missed weak atoms
Graphical-model sampling	Dependencies factorise according to graph structure	Sampling approximates distribution/inference	Equivalent parameterisations may exist	Mixing can be slow; rare evidence is difficult								
Standard PCA	Orthogonal components ordered by variance	Maximum projected variance	Sign of components arbitrary	Variance is scale-sensitive and not independence								
ICA	Observed mixtures of statistically independent, usually non-Gaussian sources	Non-Gaussianity / independence criterion	Permutation and scale/sign ambiguity	May fail with Gaussian sources or noisy/mismatched assumptions								

B. Match techniques to clinical problems (2 Marks)

Problem	**Recommended technique and condition**	**Numerical support / failure case**
Detect a slowly changing sensor trend	SFA; the underlying trend must be temporally slower than nuisance signals	Illustrative $\Delta(y_1)=0.2370$ versus $\Delta(y_2)=0.4519$ at noise SD 0.5. Strong drift can invalidate the assumption.
Shared factor with unequal noise	Factor Analysis; Gaussian linear factors and diagonal sensor-specific noise	Illustrative FA uniqueness estimates [0.3328, 0.6379, 0.5125, 0.9527]. Nonlinear factors or correlated residuals can break it.
Brief sparse abnormalities	Sparse coding / ISTA; anomalies should be represented by few dictionary atoms	At $\lambda=0.2$, the example code has 2 nonzero coefficients. Too large λ misses weak events.
Generate dependent room-state scenarios	Directed Bayesian network for causal/topological sampling or Gibbs for an undirected model	Illustrative P(Alert=High)=0.4035; large graphs or rare evidence increase sampling cost.
Compress correlated sensors	PPCA; approximate Gaussian data and isotropic residual noise	M=2 retains 74.4105% of illustrative variance. Unequal noise favours FA.

C. Diagnose a faulty PCA conclusion (1 Mark)

The conclusion is flawed because numerical variance depends on measurement units. A temperature variable measured in degrees and oxygen saturation measured as a percentage cannot be compared directly by raw variance. The high-variance variable is not automatically the dominant physiological cause. Standardise each feature, inspect correlations and loadings, validate the interpretation against domain knowledge, and consider FA if sensor-specific noise differs.

D. Relationships among models (1 Mark)

As PPCA $\sigma^2 \to 0$, its retained subspace approaches the PCA subspace, but exact zero noise can make covariance calculations ill-conditioned. FA reduces to a PPCA-like noise model when all uniqueness values are equal, but

FA's general unequal-noise flexibility is lost. For an orthonormal dictionary, sparse coding approaches ordinary linear projection as $\lambda \to 0$; however, the resulting coefficients may no longer be sparse and noise can be represented.

E. ICA and standard PCA check (included in 6 Marks)

PCA finds orthogonal directions of maximum variance and does not require statistical independence. ICA seeks statistically independent latent sources and typically assumes a linear mixture, independent sources, and at most one Gaussian source. For a reproducible check, generate two independent non-Gaussian sources, mix them linearly, standardise the mixtures, run FastICA and compare absolute correlations of recovered components with the original sources. PCA can decorrelate the mixture but generally cannot fully separate statistically dependent mixtures.

```
import numpy as np\ from sklearn.decomposition import PCA, FastICA\ from sklearn.preprocessing import StandardScaler\ \ rng = np.random.default_rng(83)\ S = np.column_stack([rng.laplace(size=2000), rng.uniform(-1, 1, size=2000)])\ A = np.array([[1.0, 0.5], [0.4, 1.0]])\ X = S @ A.T\ Xz = StandardScaler().fit_transform(X)\ P = PCA(n_components=2).fit_transform(Xz)\ I = FastICA(n_components=2, random_state=83, max_iter=2000, tol=1e-4).fit_transform(Xz)\ def best_abs_corr(A, B):\ return np.abs(np.corrcoef(A.T, B.T)[:A.shape[1], A.shape[1]:])\ print("PCA/source absolute correlations:\n", best_abs_corr(P, S))\ print("ICA/source absolute correlations:\n", best_abs_corr(I, S))
```

ICA is successful when recovered components have high absolute correlation with different true sources, allowing for permutation and sign changes. It can fail when sources are Gaussian, too noisy, dependent, or when the mixing model is not linear.

TASK 9 – Re-analyse and Adapt the Methods at Hospital Scale (8 Marks)

A. Impact analysis (2 Marks)

Missing readings challenge complete-data Gaussian estimation; use an appropriate imputation method or likelihood methods that handle missing values and retain missingness indicators. Unequal noise violates PPCA's isotropic-noise assumption, so FA or a heteroscedastic model is preferable. Sensor drift is dangerous for SFA because the algorithm treats slowly changing signals as desirable slow features, even when the change is sensor fault rather than physiology. Rare abnormalities motivate sparse coding and robust anomaly detection. A 250-node dependency graph makes exact enumeration impossible and can slow Gibbs mixing because there are 2^{250} states and strong couplings produce correlated samples.

B. Evaluate the original approach (2 Marks)

For $d=250$ and $M=10$, PPCA free parameters excluding the mean and accounting for rotations are $dM - M(M-1)/2 + 1 = 2{,}456$. FA has $dM + d - M(M-1)/2 = 2{,}705$ parameters. A full covariance matrix has $d(d+1)/2 = 31{,}375$ parameters.

Model	Free parameters	Parameters / 1,440 daily samples
PPCA (M=10)	2,456	1.7056
FA (M=10)	2,705	1.8785
Full covariance	31,375	21.7882

The parameter-to-sample ratios indicate that a full covariance model is especially prone to overfitting with only 1,440 samples per day. Even PPCA/FA require regularisation, validation across time and careful treatment of nonstationarity.

A 250-node binary graph has $2^{250} \approx 1.809 \times 10^{75}$ possible joint states. One Gibbs sweep updates each node once, so it requires 250 node updates; exact enumeration is not feasible.

C. Adapt or redesign (2 Marks)

Stage	Input	Method / action	Output
1. Data validation	Raw sensor streams	Check timestamps, ranges, duplicates and missingness; flag sensor failures	Quality report and cleaned data
2. Missing-data handling	Cleaned streams with gaps	Use time-aware imputation or missing-data likelihood; retain missingness flags	Completed/annotated matrix
3. Scaling	Validated measurements	Robust scaling or standardisation fitted on training period only	Comparable feature matrix
4. Latent factors	Scaled sensor matrix	Use FA for unequal noise; compare PPCA as a compact baseline	Latent scores, loadings and uncertainty
5. Drift handling	Time-series latent features	Detect/calibrate drift, segment regimes and apply SFA only after drift checks	Slow features with drift flags
6. Sparse events	Residuals after common factors	Apply robust residual thresholds or ISTA sparse coding with validated λ	Sparse anomaly scores
7. Dependency graph	State/dependency structure	Use sparse graph structure, blocked/parallel sampling where valid, monitor convergence	Synthetic state samples and diagnostics
8. Validation	Predictions and samples	Time-based holdout, calibration, anomaly recall, reconstruction error and sampler diagnostics	Performance report and deployment decision

Retain the latent-variable approach but modify the pipeline: use FA or robust PPCA for shared factors, explicit missing-data treatment, drift checks before SFA, sparse residual detection for rare events and graph-structured sampling with convergence diagnostics. For a 250-node graph, consider sparse dependencies and blocked or more efficient sampling methods rather than exact enumeration.

D. Evidence-based defence (2 Marks)

The redesign is supported by numerical evidence: (1) M=2 PPCA retains about 74.4105% of illustrative variance with only 8 free parameters under the stated counting convention; (2) FA estimates unequal uniqueness values, including approximately 0.9527 for S4, showing that equal noise is questionable; (3) SFA recovers the slow/fast combinations at noise SD 0.5 with slowness about 0.2370 versus 0.4519, but recovery correlation declines to roughly 0.46 at noise SD 2.0; (4) sparse coding at λ=0.2 produces two nonzero coefficients in the worked example, while a large λ can remove all coefficients; (5) a 250-node graph has about 1.81×10^{75} binary states, ruling out enumeration..

Conclusion

No single model solves every monitoring problem. PPCA provides compact probabilistic compression, FA models unequal sensor noise, SFA extracts slowly changing signals, sparse coding highlights events represented by few atoms, and graphical models represent dependencies for inference or sampling. A reliable hospital system should select methods based on their assumptions, validate them with numerical evidence, and monitor missing data, noise, drift and sampling convergence.